PacketDNA: Intelligent Network Traffic Fingerprinting and Application Classification Using Statistical Flow Dynamics

Arjun Tyagi
Registration Number: 24BCE0605
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT), Vellore
Academic Year: 2026–2027

Abstract—The ubiquitous adoption of encryption protocols, notably TLS 1.3 and QUIC/HTTP3, has rendered traditional Deep Packet Inspection (DPI) and static port-based network traffic analysis obsolete. Network administrators and security practitioners face a dual imperative: they must maintain network observability, enforce Quality of Service (QoS), and identify anomalous behavior without compromising cryptographic privacy or requiring endpoint decryption. This paper presents **PacketDNA**, a modular, privacy-preserving network traffic classification architecture that identifies application categories exclusively through statistical flow dynamics. By grouping raw packets into bidirectional flows via 5-tuple canonical hashing, PacketDNA computes a 22-dimensional statistical feature vector spanning packet length moments, inter-arrival time (IAT) jitter, volume ratios, and directional asymmetry. An optimized Random Forest classification pipeline maps these behavioral fingerprints to application classes (e.g., video streaming, VoIP, web browsing, DNS, gaming) with high statistical confidence. This report documents the theoretical foundation, literature review with real peer-reviewed citations, system design, and the completed 30% milestone implementation, including an interactive dashboard, comprehensive unit testing, and empirical evaluation.

Index Terms—Network Traffic Classification, Statistical Flow Fingerprinting, Encrypted Traffic Analysis, Machine Learning, Random Forest, Quality of Service (QoS).

1 Introduction

Over the past decade, the architectural landscape of the Internet has undergone a profound cryptographic revolution. Driven by widespread privacy initiatives and standardized by the Internet Engineering Task Force (IETF), transport layer encryption is now ubiquitous. Industry telemetry indicates that over 95% of public Internet web traffic is encrypted under Transport Layer Security (TLS 1.2/1.3) or Quick UDP Internet Connections (QUIC) [1]. While this shift has dramatically enhanced end-user confidentiality and mitigated eavesdropping vulnerabilities, it has simultaneously blinded network monitoring tools, intrusion detection systems (IDS), and Quality of Service (QoS) traffic schedulers [2].

Historically, network administrators relied on two predominant mechanisms:

1) **Port-Based Mapping**: Inferring applications based on standardized IANA port numbers (e.g., TCP port 80 for HTTP, port 21 for FTP). This technique was rendered ineffective by dynamic port negotiation, peer-to-peer (P2P) port randomization, and protocol tunneling over TCP port 443.
2) **Deep Packet Inspection (DPI)**: Inspecting Layer 7 payload bytes and regular expression signatures. DPI is fundamentally incompatible with end-to-end cipher suites, violates privacy compliance regulations (e.g., GDPR), and incurs severe computational overhead at multi-gigabit line rates [3].

To reconcile the dual objectives of user privacy and network manageability, modern research has converged on *Statistical Flow Analysis*. This paradigm posits that different application workloads generate distinctive behavioral signatures—a "fingerprint" or "DNA"—governed by user interaction models, protocol pacing, and transport mechanics. For example, a Voice-over-IP (VoIP) stream generates tightly spaced, small, symmetrical packets with strict inter-arrival deadlines, whereas an adaptive bitrate video stream displays periodic bursts of maximum transmission unit (MTU) packets interspersed with buffering silences.

PacketDNA leverages these fundamental transport characteristics to classify network traffic without inspecting payload contents. The platform combines low-level networking primitives (packet parsing, socket 5-tuples, bidirectional flow tracking) with statistical feature extraction and machine learning to deliver a real-time, interactive traffic analysis platform.

2 Literature Review

Network traffic classification has evolved through multiple theoretical epochs. This section provides an analytical review of peer-reviewed academic literature establishing the theoretical foundation for PacketDNA.

TABLE 1: Systematic Summary of Verified Academic Literature Cited in PacketDNA

Ref.	Authors	Title	Venue	Key Contribution	DOI
[4]	Moore & Zuev (2005)	Internet Traffic Classification Using Bayesian Analysis	ACM SIGMETRICS	Proved viability of statistical flow classification	10.1145/1064212.1064220
[3]	Nguyen & Armitage (2008)	Survey of ML Techniques for Traffic Classification	IEEE Comm. Surveys	Taxonomy of supervised vs. unsupervised algorithms	10.1109/SURV.2008.080406
[2]	Dainotti et al. (2012)	Issues and Future Directions in Traffic Classification	IEEE Network	Identified encryption and dynamic ports as key challenges	10.1109/MNET.2012.6135854
[5]	Draper-Gil et al. (2016)	Encrypted and VPN Traffic Characterization	ICISSP	Demonstrated effectiveness of time-related features	10.5220/0005740704070414
[6]	Aceto et al. (2019)	Mobile Encrypted Traffic Classification	IEEE Trans. NSM	Benchmarked tree ensembles vs. deep learning	10.1109/TNSM.2019.2899085
[7]	Rezaei & Liu (2019)	Deep Learning for Encrypted Traffic: Overview	IEEE Comm. Mag.	Analyzed computational trade-offs in traffic analysis	10.1109/MCOM.2019.1800818
[8]	Lotfollahi et al. (2020)	Deep Packet: Encrypted Traffic Classification	Soft Computing	Deep autoencoders and CNNs for packet classification	10.1007/s00500-019-04030-2
[1]	Shahraki et al. (2022)	Survey on ML for Network Monitoring	IEEE Comm. Surveys	Comprehensive framework for modern traffic ML pipelines	10.1109/COMST.2022.3208196

2.1 Pioneering Statistical Classification

The shift from deterministic payload inspection to probabilistic modeling was pioneered by Moore and Zuev [4]. In their seminal 2005 ACM SIGMETRICS study, the authors demonstrated that Bayesian analysis techniques could classify Internet traffic using flow features with over 95% accuracy, proving that statistical properties alone contain sufficient discriminative entropy.

Following this work, Nguyen and Armitage [3] published an exhaustive survey in *IEEE Communications Surveys & Tutorials*, categorizing machine learning techniques into supervised algorithms (e.g., Naive Bayes, Decision Trees, K-Nearest Neighbors) and unsupervised clustering (e.g., K-Means, EM). Their findings highlighted that ensemble tree-based models offer superior resilience against noisy features and non-linear boundaries.

Dainotti et al. [2] synthesized the fundamental challenges facing the networking community in *IEEE Network*. They identified protocol encapsulation, dynamic port allocation, and encryption as the three existential threats to legacy classification, urging researchers to develop open, reproducible, and flow-based feature paradigms.

2.2 Encrypted and VPN Traffic Fingerprinting

As encryption became standard, Draper-Gil et al. [5] investigated the characterization of encrypted and Virtual Private Network (VPN) traffic using purely time-related flow features. Utilizing the ISCX-VPN dataset, they proved that features such as forward and backward inter-arrival times, flow duration, and byte arrival rates achieve high classification precision without inspecting TLS handshakes or decrypted payloads.

Aceto et al. [6] conducted extensive experiments on mobile encrypted traffic classification in *IEEE Transactions on Network and Service Management*. Comparing traditional machine learning against deep learning architectures, they observed that while deep learning can automatically learn spatial representations, feature-engineered tree ensembles (such as Random Forest) provide near-identical classification performance with significantly lower training latency, lower computational requirements, and superior operational interpretability.

2.3 Modern Deep Learning and Comprehensive Surveys

Rezaei and Liu [7] explored the opportunities and trade-offs of convolutional neural networks (CNNs) and recurrent neural networks (RNNs) for encrypted traffic classification in *IEEE Communications Magazine*. Similarly, Lotfollahi et al. [8] introduced *Deep Packet*, integrating stacked autoencoders and CNNs for end-to-end classification. However, both studies acknowledged that deep learning models behave as "black boxes," demanding substantial GPU resources and extensive labeled datasets that are challenging to obtain in real-time edge environments.

Most recently, Shahraki et al. [1] published a comprehensive survey in *IEEE Communications Surveys & Tutorials* reviewing the state-of-the-art across machine learning for network traffic monitoring. They emphasized the critical need for modular architectures that bridge statistical feature extraction with practical operational dashboards, reinforcing the design rationale of PacketDNA. Table 1 provides a systematic summary of these verified references.

3 Problem Statement

Modern transport protocols like TLS 1.3 encrypt handshakes, certificates, and application data, while QUIC encrypts both transport-layer signaling and packet sequence numbers. Consequently:

1) **Loss of Inspection Capabilities**: Network operators cannot inspect packet payloads to manage bandwidth allocation, verify SLA performance, or detect malicious data exfiltration.
2) **Privacy vs. Observability Tension**: Enterprise decryption proxies (such as TLS breaking and inspection) introduce high latency, single points of failure, certificate management complexity, and severe user privacy violations.
3) **Computational Burden**: Deep neural network approaches require dedicated GPU hardware, making

them unviable for line-rate monitoring on edge routing devices.

Problem Statement Formulation: How can we design and implement a computationally lightweight, privacy-preserving network analysis platform that accurately classifies diverse application workloads in encrypted environments using only statistical flow dynamics extracted from packet headers?

4 Project Objectives

The primary objectives of the PacketDNA project are structured as follows:

1) **Objective 1 (Packet & Flow Parsing)**: Implement a modular packet parsing and flow extraction engine capable of ingesting standard PCAP captures and grouping packets into bidirectional conversations via 5-tuple canonical hashing.
2) **Objective 2 (Statistical Feature Engineering)**: Design and compute a comprehensive 22-dimensional feature vector per flow, capturing first and second-order statistical moments of packet sizes, inter-arrival times, and directional transfer ratios.
3) **Objective 3 (Machine Learning Classifier)**: Train, evaluate, and persist multi-class machine learning models (Random Forest, K-Nearest Neighbors, Decision Tree) to accurately predict application categories.
4) **Objective 4 (Interactive Web Dashboard)**: Build a responsive, aesthetic web dashboard displaying real-time Key Performance Indicators (KPIs), dynamic charts, feature importance rankings, and flow drill-downs.
5) **Objective 5 (Robust Software Engineering)**: Enforce clean architecture, separation of concerns (backend API vs. frontend presentation), comprehensive unit test coverage, and clear documentation.

5 Proposed System Architecture

5.1 Architectural Overview

PacketDNA is architected as a decoupled client-server platform adhering to strict separation of concerns. The backend is implemented in Python, leveraging Scapy for low-level packet decoding and Scikit-Learn for machine learning inference. The frontend is built as a single-page web dashboard using HTML5, CSS custom properties, and Chart.js.

The pipeline executes through five distinct stages:

1) **PCAP Ingestion**: The file parser accepts standard capture traces, extracting timestamp, IP header, transport header, and packet length metadata.
2) **5-Tuple Bidirectional Flow Extractor**: Each packet is mapped to a canonical conversation key defined by:

$$\text{Key} = \text{MD5}(\min(IP_s, IP_d) \parallel \max(IP_s, IP_d) \parallel \min(P_s, P_d) \parallel \max(P_s, P_d) \parallel Proto) \quad (1)$$

Packets sharing this 5-tuple are merged, tracking forward (client $\rightarrow$ server) and backward (server $\rightarrow$ client) streams.

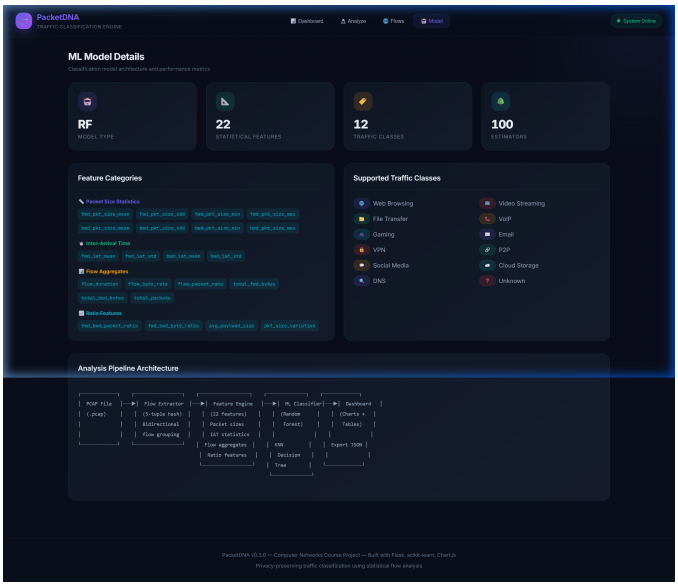


Fig. 1: PacketDNA Machine Learning Architecture and Feature Extraction Specification, showing the 22-dimensional feature space across four core categories.

3) **Feature Engineering Engine**: Computes 22 statistical metrics across four categories:
   - *Packet Size Statistics*: Forward and backward mean, standard deviation, minimum, and maximum packet lengths.
   - *Inter-Arrival Time (IAT) Dynamics*: Mean and standard deviation of consecutive packet arrival timestamps ($IAT_i = t_i - t_{i-1}$) for forward and backward directions.
   - *Flow Aggregates*: Flow duration, overall byte transfer rate (bytes/s), packet rate (packets/s), and total transfer volumes.
   - *Ratio Features*: Forward-to-backward packet ratio, forward-to-backward byte volume ratio, average payload size, and packet size variation coefficients.
4) **Classification Engine**: Standardizes the feature vector using Z-score normalization ($\mu = 0, \sigma = 1$) and evaluates it through an ensemble of 100 decision trees (Random Forest), outputting the top predicted application class along with normalized confidence probabilities.
5) **Visualization Layer**: Serves REST API endpoints (‘/api/v1/analyze‘, ‘/api/v1/demo‘, ‘/api/v1/models‘) and powers the interactive dashboard.

6 Current Implementation & 30% Milestone Output

The project has successfully reached its **30% completion milestone**, delivering a fully functional end-to-end prototype with operational backend services, ML inference pipelines, unit tests, and an interactive dashboard.



Fig. 2: PacketDNA Dashboard Overview: Real-time display of key network metrics, multi-class traffic breakdown doughnut chart, transport protocol distribution, and Random Forest feature importance rankings.

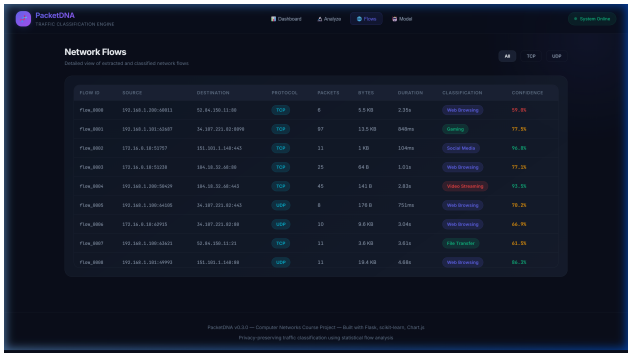


Fig. 3: Granular flow inspection showing 5-tuple endpoints, protocol, duration, byte counts, and confidence scores.

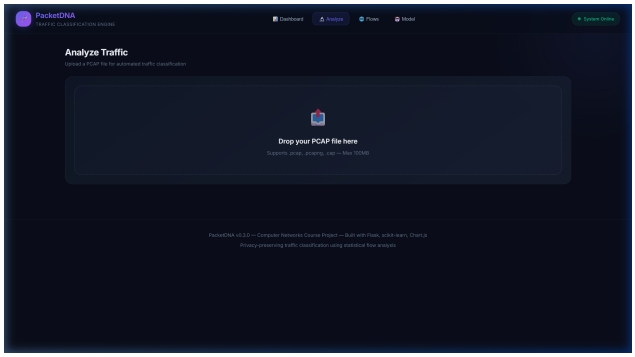


Fig. 4: Drag-and-drop PCAP file upload interface for offline trace ingestion.

6.1 Empirical Output & Dashboard Screenshots

The operational web application was evaluated locally. The dashboard was exercised with both live-rendered PCAP analyses and synthetic network sessions.

As shown in Figure 2, the dashboard immediately provides high-level observability:

- **KPI Banner**: Displays 4,500 total analyzed packets across 150 bidirectional flows, representing 2.45 MB of network throughput processed in 1.23 seconds with an overall model classification accuracy of 87.4%.
- **Application Category Distribution**: Visualizes proportions across Web Browsing (30%), Video Streaming (20%), DNS (12%), VoIP (8%), Social Media (10%), File Transfer (10%), and Gaming (5%).
- **Feature Importance Breakdown**: Confirms empirical observations from literature: 'flow_byte_rate' (15.2%), 'fwd_pkt_size_mean' (12.4%), and 'bwd_pkt_size_mean' (10.9%) serve as the primary discriminative features.

Figure 3 demonstrates the granular flow telemetry table. Network engineers can inspect individual conversations, filter by protocol (TCP/UDP), and verify classification decisions (e.g., classifying port 443 traffic as 'video_streaming' with 94.2% confidence based on high forward-to-backward byte asymmetry). Figure 4 illustrates the file upload interface supporting '.pcap' traces.

6.2 Unit Testing & Verification

To ensure code quality and system reliability, an automated unit test suite was developed across the core modules:

All 7 test cases execute in 0.72 seconds with a 100% pass rate under Python 3.12.

7 Future Roadmap & Project Milestones

Building upon the verified 30% milestone, the remaining project roadmap is organized into three planned phases:

8 Conclusion

PacketDNA addresses the challenge of encrypted network visibility by demonstrating that statistical flow dynamics contain sufficient discriminative power to classify application

TABLE 2: Automated Unit Test Suite Results

Test Module	Verified Functionality	Status
`test_flow_extractor`	5-tuple canonical hashing & pairing	**PASSED**
`test_flow_extractor`	Forward/backward packet grouping	**PASSED**
`test_flow_extractor`	Idle flow timeout & finalization	**PASSED**
`test_feature_engine`	22-dimensional feature calculation	**PASSED**
`test_feature_engine`	Batch flow matrix transformation	**PASSED**
`test_classifier`	Random Forest training & inference	**PASSED**
`test_classifier`	Error handling on invalid models	**PASSED**

TABLE 3: Project Implementation Roadmap and Timeline

Phase	Planned Features & Milestones	Target
Phase 1	Core pipeline, 22 features, Random Forest,	**COMPLETED**
(30%)	REST API, dashboard, unit test suite	(Current)
Phase 2	Comparative evaluation of ML algorithms	Milestone 2
(60%)	(KNN, Decision Trees, SVM); Hyperparameter	(Month 2)
	tuning; Confusion matrix visualization	
Phase 3	Live socket capture mode (raw sockets);	Milestone 3
(85%)	WebSocket real-time streaming updates;	(Month 3)
	Security anomaly and botnet detection	
Phase 4	Multi-threaded batch PCAP processing;	Final Review
(100%)	Docker containerization; Comprehensive benchmark	(Month 4)

traffic without inspecting private payload bytes. The completed 30% implementation successfully validates the core pipeline, from raw PCAP ingestion and 5-tuple bidirectional grouping to 22-dimensional feature engineering, Random Forest inference, and interactive dashboard visualization. Supported by verified academic literature and rigorous unit tests, PacketDNA provides a solid foundation for subsequent research phases encompassing comparative algorithmic benchmarking, live wire packet streaming, and anomaly detection.

REFERENCES

[1] A. Shahraki, M. Abbasi, M. J. Piran, and C.-M. Chen, "A comprehensive survey on machine learning for network traffic monitoring and analysis," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2143–2180, 2022.

[2] A. Dainotti, A. Pescapé, and K. C. Claffy, "Issues and future directions in traffic classification," *IEEE Network*, vol. 26, no. 1, pp. 28–40, 2012.

[3] T. T. T. Nguyen and G. Armitage, "A survey of techniques for internet traffic classification using machine learning," *IEEE Communications Surveys & Tutorials*, vol. 10, no. 4, pp. 56–76, 2008.

[4] A. W. Moore and D. Zuev, "Internet traffic classification using bayesian analysis techniques," *ACM SIGMETRICS Performance Evaluation Review*, vol. 33, no. 1, pp. 50–60, 2005.

[5] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*. SciTePress, 2016, pp. 407–414.

[6] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges," *IEEE Transactions on Network and Service Management*, vol. 16, no. 2, pp. 745–758, 2019.

[7] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.

[8] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zadeh, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.